

# YuanRong: A Production General-purpose Serverless System for Distributed Applications in the Cloud

Qiong Chen, Jianmin Qian, Yulin Che, Ziqi Lin, Jianfeng Wang, Jie Zhou, Licheng Song  
Yi Liang, Jie Wu, Wei Zheng, Wei Liu, Linfeng Li, Fangming Liu\*, and Kun Tan  
Huawei Technologies Co., Ltd

## ABSTRACT

We design, implement, and evaluate *YuanRong*, the first production general-purpose serverless platform with a unified programming interface, multi-language runtime, and a distributed computing kernel for cloud-based applications. *YuanRong* addresses many limitations of existing *Function-as-a-Service (FaaS)* systems, particularly in performance and lack of important features. First, our fast function system supports sub-millisecond function invocation and locality-aware hierarchical scheduling. Second, our multi-semantic built-in data system achieves object exchange latency of 200 microseconds, enabling end-to-end latency of 2 milliseconds for streaming elements at 5Gbps throughput. Third, the extensible, portable *Service Bridge* bridges stateless and stateful operations, allowing connection reuse and distributed transactions, and offers unified back-end abstraction for multi-cloud portability. *YuanRong* has been deployed for over 3 years at Huawei across nearly 20 datacenter regions, processing up to 30 billion requests per day on more than 100,000 CPU cores, with a daily average CPU usage of 53%. It serves various serverless workloads, including enhanced FaaS services, microservices, data analytics, deep model training and serving, and some HPC workloads. Our experience shows that Spring-based microservices can be migrated to *YuanRong* within one day, reducing resource costs by 90%, demonstrating its generality and efficiency in supporting a broad spectrum of applications.

## CCS CONCEPTS

• **Computer systems organization** → **Architectures; Distributed architectures; Cloud computing;**

## KEYWORDS

FaaS, serverless, cloud computing, distributed systems

### ACM Reference Format:

Qiong Chen, Jianmin Qian, Yulin Che, Ziqi Lin, Jianfeng Wang, Jie Zhou, Licheng Song, Yi Liang, Jie Wu, Wei Zheng, Wei Liu, Linfeng Li, Fangming Liu\*, and Kun Tan. 2024. YuanRong: A Production General-purpose Serverless System for Distributed Applications in the Cloud. In *ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24)*, August 4–8, 2024, Sydney, NSW, Australia. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3651890.3672216>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

ACM SIGCOMM '24, August 4–8, 2024, Sydney, NSW, Australia

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0614-1/24/08

<https://doi.org/10.1145/3651890.3672216>

## 1 INTRODUCTION

Serverless computing is emerging as the next-generation cloud computing paradigm, offering attractive features such as ease of programming, rapid elasticity, fine-grained billing, and NoOps. Based on these promising prospects, many major cloud providers have rolled out their Function-as-a-Service (FaaS) offerings, such as AWS Lambda [35], Google Cloud Functions [19], and Microsoft Azure Functions [20], which are complemented by an ecosystem of Backend-as-a-Service (BaaS) offerings such as object storage (OBS), databases, catering to specific application requirements. Put simply, serverless computing is generally known as FaaS + BaaS.

Although these offerings are being rapidly adopted by tenants [43], they are currently dominated by simple event-driven request handler applications ranging from web services to Internet-of-Things [48]. Typically, when a request arrives, a function instance encapsulated in a container is launched to serve the request and usually does not need to coordinate with other instances. Recently, there are emerging significant interests in exploiting serverless computing for more general and complex applications, in which multiple function instances are coordinated to complete a task, and states are needed to share across various instances, such as video processing [3, 17], data analytics [32, 33, 42, 46], and machine learning [8, 27, 56]. However, existing FaaS platforms fail short to support these general-purpose applications due to low performance and lack of important features. We summarize their limitations as follows:

**Lack of Fine-grained Coordination (L1).** To extend support for general-purpose applications, coordination between functions is required. For example, if function A uses the output of function B, there must be a way for A to know when its input is available, even if A and B are on the same nodes. This notification should be promptly, especially when there are multiple function instances are involved. However, current FaaS platforms are incapable for such fine-grained coordination, since functions are unaddressable in the current design and there is no way to pass a message from one function instance to another, as well as the lack of data locality-aware scheduling control. All coordination among functions is either implemented through external storage or a backend service (e.g., SQS), which introduces significant latency and is costly when coordination occurs frequently.

**Rely on Remote Storage for State & Data Transfer (L2).** Distributed cloud applications often require ephemeral state or intermediate data sharing between their components, such as application-wide caches of complex microservices [26, 28], or intermediate results of data analytics [32, 33]. However, the existing FaaS offering does not support state storage. In microservices or batch processing, all states or application intermediate data has to be

\*Corresponding author.

†This work does not raise any ethical issues.

stored in some remote storage services, e.g., OBS, which introduces significant latency (i.e., hundreds of milliseconds [23]), or low throughput due to limited bandwidth provision of remote storage services. In streaming processing, unbounded exchanged data are batched within a window, and execution is triggered based on the window size or a time interval. On conventional FaaS platforms, developers have to invoke functions or rely on BaaS queue services to exchange streaming data, which incur network traffic and memory copy overhead.

**Limited Interactions between Stateless & Stateful (L3).** Supporting general-purpose applications also complicates the interaction between stateless functions and backend services. Backend services are typically stateful and maintain authenticated and active connections for each client, e.g., a JDBC connection. These connection states are difficult to be shared among collaborated function instances. Even though one function could pass backend connection information to another function, concurrent operations on a BaaS could still be hard, as the two function instances are essentially performing the transaction in a distributed fashion.

**Lack of Unified API for Diversified BaaS (L4).** One important requirement from most application developers is the ability to deploy their applications on multiple clouds. While most cloud providers offer similar backend functionalities, the detailed interfaces of these services are largely different, e.g., AWS S3 versus Azure Storage. Even a single cloud provider often offers hundreds of BaaS to meet the needs of different scenarios. It is cumbersome and costly for application developers to integrate multiple similar but different SDKs to access backend services in different cloud environments. Ironically, the fear of potentially being locked in is another major reason for application developers to opt for serverless.

Previous arts may address some of the aforementioned limitations [2, 37, 44, 61]. But to our best knowledge, none of the existing serverless platforms have addressed all these issues in a holistic way [47]. This paper presents the design, implementation, and evaluation of YuanRong\*, the first production serverless platform which fills the gap left by existing serverless platforms in efficiently supporting *general-purpose workloads*. YuanRong provides a unified open serverless interface to facilitate cloud-based application development, and also employs a distributed computing kernel to address the aforementioned limitations. To address L1, we build a fast function system that supports sub-millisecond cross-function invocation latency by simplifying components communication path. Further, we develop a locality-aware hierarchical scheduler to efficiently track and schedule hundreds of thousands of function instances even in highly dynamic situations. To address L2, we implement a multi-semantic data system that provides a low-latency, high-throughput data cache through a distributed shared memory pool to efficiently support state management and data exchange among function instances. We further develop a portable framework called *Service Bridge* to address L3 and L4. It provides a set of unified interfaces for diversified BaaS like OBS, SQL database, and allows plug-in *bridge functions* to be installed, either by the application developer or cloud provider, to intercept requests from the unified interface and translate them into specific requests to the

cloud-provided backends. This way, Service Bridge effectively enables backend connection reuse and distributed transactions among collaborated functions. We believe providing a unified backend interface abstraction is potentially a way to mitigate vendor lock-in.

YuanRong has been widely deployed at Huawei for its internal cloud-native services and also an enhanced FaaS service for external customers, serving over 30B requests per day on more than 100,000 CPU cores with daily average CPU usage around 53%. It serves various serverless workloads in our cloud, ranging from microservices, data analytics, deep model training and serving, as well as some HPC workloads. By making applications serverless, it greatly simplifies application development and results in a shorter time-to-market. Our experience convinces us that YuanRong is general and highly efficient to support a large spectrum of applications.

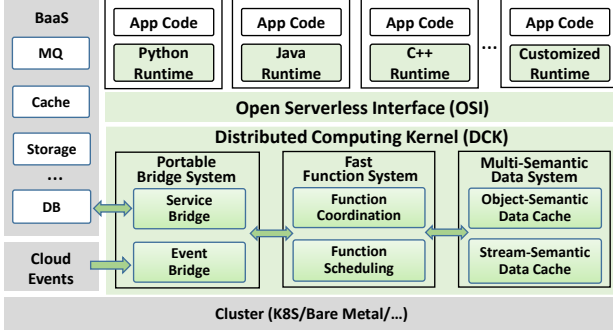
The contributions of this paper are summarized as follows.

- We design and implement YuanRong, which to our best knowledge is the first production general-purpose serverless platform with a unified interface, a multi language runtime, and a distributed computing kernel.
- We develop a fast, scalable function system that supports sub-millisecond function-to-function invocation and achieves 2× lower scheduling latency than current centralized scheduling mechanism through hierarchical distributed scheduling.
- We develop the first multi-semantic, built-in data system that achieves 200 microseconds object exchange latency (orders of magnitude faster than Redis), and enables 2 milliseconds streaming element end-to-end latency at 5Gbps throughput.
- We develop the first extensible, portable Service Bridge that bridges stateless and stateful to enable connection reuse and distributed transactions, and provides unified API abstraction to achieve multi-cloud portability.
- We deploy YuanRong in 20 datacenter regions over 3 years. Our experience shows that Spring framework based microservices can be migrated to YuanRong within 1 day and resource costs are reduced by 90%.

## 2 DESIGN OVERVIEW & REQUIREMENTS

This section overviews the design of YuanRong. The overall architecture of YuanRong is shown in Fig. 1. YuanRong exposes a unified programming interface to interact with function instances. We call this interface as *Open Serverless Interface (or OSI)*. Hereafter in this paper, we use YuanRong API or YuanRong OSI to refer to this interface. YuanRong OSI is implemented with C/C++ and exposes standard C calling conventions, but functions can be developed in various popular programming languages, such as Java, JavaScript, or Python. Therefore, for each such language, we also provide a language-specific runtime that is integrated together with application code in container images. The core part of YuanRong is the underlying *distributed computing kernel (DCK)* that implements OSI. DCK contains three major subsystems: a fast function system, a multi-semantic data system, and a portable bridge system. YuanRong offers a holistic system that automatically manages and optimizes all stages of the function execution after deployment while supporting bare metal processes or existing container orchestrators (e.g., Kubernetes) to deploy and orchestrate applications.

\*YuanRong means giant chariot in ancient China.



**Figure 1: The overall architecture of YuanRong that contains three major parts: 1) multi-language runtimes; 2) open serverless interface; and 3) distributed computing kernel.**

## 2.1 Open Serverless Interface

Table 1 summaries YuanRong OSI that contains APIs in three categories:

- **Function.** Function API is used to manage function lifecycle (e.g., create or terminate a function), coordinate functions with one another (e.g., invoke), and manage the internal state (e.g., save states or load pre-stored states).
- **Object and Stream.** Data object and stream API defines methods for functions to interact with the data system. YuanRong data system supports shared access to named data objects or unbounded data streams.
- **Bridge.** Bridge API provides necessary methods for functions to communicate with backend services. There are Event APIs for functions to receive incoming events and Service APIs to request backend services.

## 2.2 Multi-language Runtime

YuanRong API is implemented in C/C++, but many application functions are developed using other popular programming languages like Java and Python. To facilitate function code to access YuanRong API, we also provide several language-specific runtimes. These runtimes may leverage certain language-specific features and provide more advanced interfaces to application developers.

Function is the basic unit of computing managed by YuanRong. Fig. 2 shows simple code snippets to define a remote YuanRong function and invoke this remote function from the main function in Python, Java, and C++. The developer could specify a resource hint, e.g., the number of CPU cores and memory size, to a function by setting the *options* parameters in the *invoke* interface. The *invoke* method in YuanRong API is by nature asynchronous. That is, a caller to invoke method will get a reference to a *future*, which is stored in the YuanRong data system. The callee, however, will also obtain another reference to the same future object. Therefore, the callee can *put* results into the future object, and the caller can *wait* or *get* these results via its reference. Due to page limitations, please see § A.1 for details.

## 2.3 Distributed Computing Kernel

The *distributed computing kernel* of YuanRong aims to overcome the aforementioned L1 ~ L4 limitations discussed in § 1 and should satisfy the following design requirements.

**Fast Function System.** Fine-grained function coordination in general-purpose workloads require low-latency function-to-function invocation and locality-aware scheduling. To achieve this, we design a fast function system achieves sub-millisecond invocation latency by optimizing invocation via simplifying communication between system components. We further place functions through a hierarchical distributed scheduler, and considers data locality of intermediate outputs while maintaining scalability.

**Multi-Semantic Data System.** Function state and intermediate data in general-purpose workloads also require low-latency data exchange. To achieve this, we propose a multi-semantic data system with key-value object and pub-sub stream semantics, which pools distributed memory together to form a shared memory cache. We further optimize stream semantics by employing data batching, asynchronous data pushing, and memory sharing techniques, achieving a 10Gbps throughput at a latency of few milliseconds.

**Portable Bridge System.** Sharing connections (e.g., a JDBC connection) to stateful backends among stateless functions, and multi-cloud deployment remains difficult. Therefore, we propose a portable bridge system that abstracts the communication between functions and backends. On the one hand, an *Event Bridge* provides methods for BaaS or API gateways to send events and trigger function execution; On the other hand, a *Service Bridge* provides a unified way for the function to access diversified BaaS, such as AWS S3 and Azure Storage. We also provide pre-built bridge functions and plugin mechanisms to extend diverse vendor-specific BaaS functionalities, and mitigate vendor lock-in. Note that we do not prevent application developers from directly using the BaaS API, but rather provide them with the alternative of using the unified OSI APIs to reap the aforementioned benefits of YuanRong.

## 3 FAST FUNCTION SYSTEM

In this section, we describe a fast function system that supports fine-grained function coordination and scalable function scheduling. Due to page limitations, we introduce the function metadata and lifecycle management in § A.2.

### 3.1 Fine-grained Function Coordination

YuanRong achieves sub-millisecond invocation by minimizing the function communication path, and facilitates direct function-to-function remote procedure call (RPC) by implementing a function proxy at each node. The function proxy primarily handles message forwarding on the data plane, offering high performance asynchronous calls based on the *libprocess* actor-style message passing programming model [24]. One can call *invoke* (see Table 1) interface with a function name or ID, which may send an RPC request to any existing instance with a certain load balancing rule (e.g., round-robin) by the function proxy, or a new instance may be created. One particular feature of *invoke* is that it allows passing parameters *by reference*. This is because the data system is shared among all function instances, allowing references to objects stored in the data system to be directly passed from one function to another. The

**Table 1: YuanRong OSI APIs.** We divide the APIs into three categories: 1) Function API allows functions to manage function lifecycle; 2) Data object and Stream API defines methods for functions to interact with the data system; and 3) Bridge API provides a set of unified interfaces for common backend services like OBS, SQL database, and messaging services.

| Categories | APIs & Description                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Function   | <b>Create:</b> Create a new function instance.                                                                       | <b>Invoke:</b> Invoke another function asynchronously.                                                                                                                                                                                                                                                                                                                                                                          |
|            | <b>Exit:</b> Self-terminate function instances.                                                                      | <b>Kill:</b> Kill function instances.                                                                                                                                                                                                                                                                                                                                                                                           |
|            | <b>SaveState:</b> Save the state of the function.                                                                    | <b>LoadState:</b> Load the state of the function.                                                                                                                                                                                                                                                                                                                                                                               |
| Object     | <b>Put:</b> Save named data object to data system.                                                                   | <b>Get:</b> Get the object value according to its name.                                                                                                                                                                                                                                                                                                                                                                         |
|            | <b>GInc, GDec:</b> Increment and decrement reference counts of named objects, respectively.                          |                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Stream     | <b>Publish, Subscribe, Close:</b> Create and destroy publish-and-subscribe (pub-sub) relationships on a stream name. |                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|            | <b>Send, Flush:</b> Send unbounded data and make it globally visible, respectively.                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|            | <b>Recv, Ack:</b> Receive unbounded data and acknowledge the consumption after its usage, respectively.              |                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Bridge     | <b>Service Bridge</b>                                                                                                | <b>CreateBridge, ReleaseBridge:</b> Create and release a bridge function for requesting backend services.<br><b>Execute, Commit, Rollback:</b> Execute SQL, commit and rollback transactions for relational database.<br><b>Set, Get, Expire:</b> Add and get cache objects, set cache expiration time for cache service.<br><b>PutObj, GetObj, DeleteObj:</b> Add, get, and delete storage objects for object storage service. |
|            | <b>Event Bridge</b>                                                                                                  | <b>CreateChanel, DeleteChanel:</b> Create and delete event channel for event subscription.<br><b>AddEvent, DelEvent:</b> Add and delete an event source on the chanel.<br><b>AddTarget, DelTarget:</b> Add and delete event target, including filter and format conversion rules.                                                                                                                                               |

```

1: @yr.invoke
2: def add_one(x):
3:     return x + 1
4:
5: objref = add_one.options(cpu=1000,
6: memory=1024).invoke(5)
7: print(yr.get(objref))

```

(a) Python

```

1: public class TaskExample {
2:     public static int addOne(int x) {
3:         return x + 1;
4:     }
5: }
6:
7: FuncHandler<String> funcHandler =
8:     YR.function(TaskExample::addOne).options(
9:         cpu=1000, memory=1024);
10: ObjectRef objRef = funcHandler.invoke(5);
11: System.out.println(YR.get(objRef));

```

(b) Java

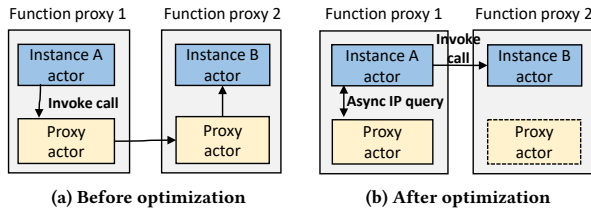
```

1: int AddOne(int x)
2: {
3:     return x + 1;
4: }
5: YR_INVOKE(AddOne)
6:
7: auto objRef =
8:     YR::Function(AddOne).Options(cpu=1000,
9:     memory=1024).Invoke(5);
10: cout << *(YR::Get(objRef));

```

(c) C++

**Figure 2: A simple code snippets written in three mainstream languages using YuanRong APIs.**



**Figure 3: Optimization of function invocation by reducing the interaction path between components.**

detailed mechanism for supporting pass-by-reference is described in § 4.1. This feature can avoid transmitting a large amount of data during invocation, and data is only pulled when the function is actually required, thus reducing invocation latency. However, the forwarding path of invocation messages remains complex. In our earlier version (Fig. 3a), when communicating across nodes, the caller instance first create an *instance actor* corresponding to the function to enable asynchronous message processing. Subsequently, the *proxy actor* in the function proxy retrieve the routing information of the instance and forward the invocation message to the proxy actor of target node, ultimately reaching the callee instance. This process involved multiple asynchronous calls, leading to longer invocation paths.

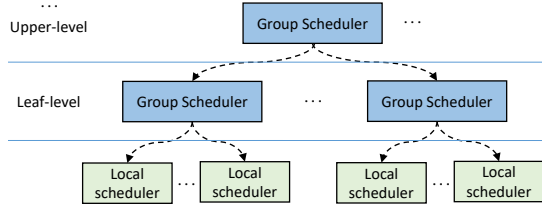
To address this issue, we optimize function invocation path by sending request directly between instance actors. As shown in Fig. 3b, the instance actor first asynchronously queries the routing

information (e.g., IP and port) of the invoked function from proxy actor. Once the routing information is obtained, subsequent invocations of the corresponding function result in the direct transfer of the invocation message to the relevant instance. This optimization eliminates the need for the proxy actor to perform multiple forwarding operations, thereby reducing invocation latency.

### 3.2 Scalable Function Scheduling

YuanRong employs a hierarchical distributed scheduler for function scheduling as depicted in Fig. 4, which effectively leverages data locality and ensures high scalability. There are two types of schedulers, *group scheduler* and *local scheduler*. The local scheduler is deployed on each node and is responsible for resource allocation and periodically synchronizing node resource usage. The group scheduler is hierarchical and organized in a tree structure. The leaf-level group scheduler is responsible for managing all local schedulers in the group, while the upper-level group scheduler is responsible for managing the lower-level group schedulers.

To minimize latency caused by cross-node communication, YuanRong strives to create function instances within a specific scope. When a function instance is created by the API gateway, i.e., to serve an external request by an entry function, the group scheduler always picks up the least busy group and deliver the request until it reaches the local scheduler. Conversely, when a function instance is created by an existing caller function, the local scheduler of the caller node is first invoked to allocate resources for the callee in the



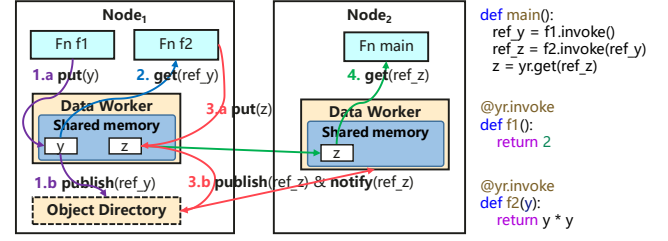
**Figure 4: Architecture of hierarchical distributed scheduler supports top-down and bottom-up scheduling.**

same node to reduce the cross-node communication overheads and transmission of intermediate outputs. If that fails, the leaf-group scheduler is invoked and a node with the least load is chosen. This way, the leaf-group scheduler tries to centralize the caller and callee in the same group to ensure data locality. If none of the nodes in the group can satisfy the resource requirement, the upper-group scheduler searches the nodes in other groups under its subordinates. This streamlined design ensures that function instances from a single invocation chain are deployed in close proximity, minimizing network and intermediate data interactions between nodes.

However, both the aforementioned top-down (external request) and bottom-up (internal request) scheduling may introduce conflicts in resource allocation. Specifically, each scheduler maintains their own view of resource allocation. This can lead to scheduling failures for function instances triggered by external requests, as the group scheduler may not be aware of the updated status of the local scheduler, resulting in insufficient resources. To address this issue, we developed a combination of *pulling*, *pushing*, and *revising* techniques to synchronize the status of resource allocation. First, *pulling* is employed, where the group scheduler retrieves the resource allocation view from the local scheduler or low-level group scheduler every second. Each pulling request includes a revision that indicates whether the group scheduler should update its cache based on the response. Second, *pushing* is utilized, wherein the local scheduler monitors resource allocation at a higher frequency (every 100 milliseconds). If there is a significant increase or decrease in the resource allocation ratio, the local scheduler promptly notifies the higher-level group scheduler to refresh its cache. Last, *revising* is employed as conflicts in resource allocation are inevitable in distributed scheduling, particularly under high concurrency conditions. Once a scheduling decision failed, revised resource allocation views are returned along with the failed function instance response. This mechanism enables the correction of outdated cache within the group scheduler, ensuring accurate resource allocation.

Moreover, network congestion can also lead to failures in creating function instances. When network traffic exceeds the configured threshold, traffic can be redirected between group schedulers to eliminate hotspots. In extreme cases where the group scheduler is unable to receive or respond to new requests due to network congestion, the local scheduler will send a request to the global scheduler (i.e., top-level group) to switch to an alternative upper-level group scheduler. Concurrently, the global scheduler will update and persist the tree structure to reflect the changes.

This hierarchical structure paves the way for a highly scalable scheduling framework for YuanRong by allowing a group scheduler at one level to be supervised by another group scheduler at a higher level. The upper-level group scheduler maintains a broader view of



**Figure 5: Architecture of object-semantic data cache. Objects are locally put and get by functions and transparently transferred across-node by data workers.**

resources with the goal of reducing resource slack or waste. Based on the resource allocation view, this group scheduler can easily add or remove nodes and also determine which lower-level group should handle the function request. This feature of YuanRong's hierarchical group scheduler achieves high scalability.

## 4 MULTI-SEMANTIC DATA SYSTEM

In this section, we present a multi-semantic built-in data system that provides a low-latency, high-throughput data cache over a distributed shared memory pool for efficient state management and data exchange between function instances.

### 4.1 Object-Semantic Data Cache

Function data exchange through cloud cache services incurs additional costs and network traffic overheads, and FaaS platform memory is often under-utilized. We thus propose utilizing the vacant distributed memory to cache intermediate across-function shared data. To support pass-by-reference function data exchange, we propose a key value object semantic, which provides *put*, *get*, and lifecycle management operations. To achieve low latency, a *data worker* is deployed on each node, functions only interact with local data workers via shared memory, and data workers transparently transfer and cache object copies (refer to Fig. 5). All data workers with cached object copies can serve as providers.

We also provide support for passing arguments by value directly through function invocation. However, when dealing with large data exchanges, we recommend opting for the pass-by-reference semantic. This approach helps avoid unnecessary movements of large data. Further, we propose overlapping computation and on-demand reference resolving by initiating an asynchronous data prefetching thread. The thread is responsible for retrieving subsequently accessed data through object *get* operations.

**Object Metadata Management.** Each object is associated with a universally unique ID (UUID). To determine object location by UUID, we introduce an *object directory* that maintains a list of object locations. Both *put* and *get* operations notify data workers to update locations on the directory. The directory selects a location from the list as data provider. When there are *get* operations for hot objects, the workloads can be balanced among data providers. We support pass-by-reference semantic by wrapping the UUID into an object reference and passing the references as function arguments.

Due to the asynchronous nature of function invocation, there are cases when an object reference is resolved, but its value has not been put yet (Fig. 5, step 4). In such cases, we rely on the object directory

to subscribe to the visibility of an object. When an unresolved object becomes globally visible, the object directory notifies the relevant data workers (Fig. 5, step 3.b). Traditional cloud cache services do not offer such subscription capabilities. Our object publishing and subscription design bridges this gap, enabling efficient construction of dynamic computation graphs with data dependencies.

**Object Consistency and Concurrency Control.** We support both mutable and immutable objects. When an object is attributed as immutable on creation, data system will reject subsequent *put* operations on the same object. As for mutable data, in the presence of multiple cached copies, simultaneous *put* and *get* operations on the same data are possible. We support two consistency models: PRAM and causal. Updates are eventually observed by functions in the PRAM model, and are immediately globally visible in the causal model. To achieve this, we send cache invalidation messages from the object directory to all data workers that cache copies of the object. These messages are sent asynchronously for the PRAM model and synchronously for the causal model. Users can specify different consistency models as input parameters for the *put* interface to balance consistency level and overhead.

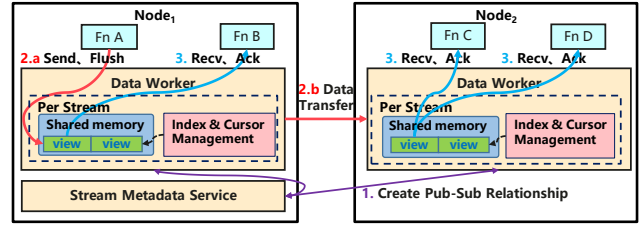
By default, write concurrency control is supported by the last-writer-win mechanism, and the object directory controls the visibility of object versions. To ensure strong consistency and data integrity, we provide global locking mechanisms. To avoid concurrent reading and writing of the same object copy in shared memory, we introduce latches. Consequently, only complete object copies are visible to readers.

**Object Scalability and Volume.** To achieve scalability, we distribute the object directory across different nodes. To minimize access latency, we choose a home directory for an object based on the location of the reference creator, where most metadata operations occur. We wrap the routing information of the home directory in the object reference and pass it in the arguments of the function calls. To reduce IPC overhead, the object directory module is integrated into the data worker. As a best practice, we recommend that developers design nested functions to avoid home node bottleneck.

To handle huge volume of data and efficiently utilize object cache, we remove cold copies from the cache and free the memory for newly added hot copies. To ensure accessibility of all objects before deletion, we design an eviction mechanism based on primary copies. Primary copies, obtained by *put* operations, are either kept in memory or swapped to local SSDs, while local copies can be deleted directly. When a primary copy is in the eviction process, it waits until swapped out to local SSDs, and stores the metadata in data worker so that the copy can be retrieved later. Due to page limitations, we describe object lifecycles and reliability in § A.3.1.

## 4.2 Stream-Semantic Data Cache

In streaming scenarios, unbounded data are batched within a window and transferred among functions. To support such scenarios, we design a stream semantic, based on a publish-subscribe (pub-sub) model. Each pub-sub relationship is associated with a designated stream name. Producers can publish data to a specified stream and send elements, while consumers can subscribe to a particular stream and receive a desired number of elements within a predefined timeout interval. To establish a one-to-many relationship, a



**Figure 6: Architecture of stream-semantic data cache.** The pub-sub relationships are built first, then streaming data are locally sent and received by functions, and data workers transfer and cache the data transparently.

source function can publish data to a named stream with multiple subscribers, thereby enabling data broadcasting. Conversely, a many-to-one relationship can be established by allowing a destination function to subscribe to a stream of multiple producers, enabling gathering data from multiple sources for subsequent reduction and aggregation processing.

To avoid costly metadata operations such as object publishing and reference counting, the pub-sub relationship is established once and subsequent data sending and receiving operations do not necessitate any metadata operations. Fig. 6 illustrates components for the stream semantic. A streaming metadata service manages the pub-sub relationships and notifies appropriate data workers to facilitate subsequent data transfer. The metadata component can be distributed in a similar way to the object directory. In subsequent data sending and receiving, to represent streaming data and optimize by batching, we design a page-based memory layout. Streaming data can be batched in shared memory pages, thereby reducing memory requirements and minimizing copy costs. To enable efficient data consumption, we design index structures for pages and elements, along with consumption cursors for subscribers.

**Buffered Data Sending.** To minimize network overhead, we introduce the concept of page-batched data sending. When a producer performs a send operation, the function appends elements to a shared memory page and buffers element metadata, such as data size. The data batching reduces the RPC overhead. In producer creation, developers can specify an interval and page size as flushing triggers. In a *flush* operation, the function synchronizes the changes to element metadata with the local data worker via IPC. Based on these changes, the data worker maintains an index to support subsequent element and page searches, and performs asynchronous data transfer to remote consumers. To maximize throughput for data transfer across nodes, we push multiple on-the-fly page to remote workers with consumers, and asynchronously process acknowledgments of page reception.

**Shared Data Receiving.** To reduce data movements, we fully make use of shared memory pages. When both producers and consumers are scheduled on the same node, elements are received via shared memory, eliminating the network traffic and copy overheads. In cases where consumers are on the same node while producers are not, we optimize the data transfer process by sending data to the consumer node only once, caching the elements in shared memory pages, and minimizing the occurrence of unnecessary RPCs.

During a *recv* operation, developers can specify the expected element size and timeout parameters to create a window for processing. When handling the *recv* operation, the data worker looks-up the index structure of elements and shares the appropriate pages and element metadata with the consumer, thus avoiding memory copies and reducing memory footprint. Once the function has processed the elements, it notifies the data worker by performing an *ack* operation, indicating that the elements are no longer referenced. The page is automatically freed when it is no longer referenced. Due to page limitations, we describe back pressure mechanism which handles variations in processing speed between producers and consumers in § A.3.2.

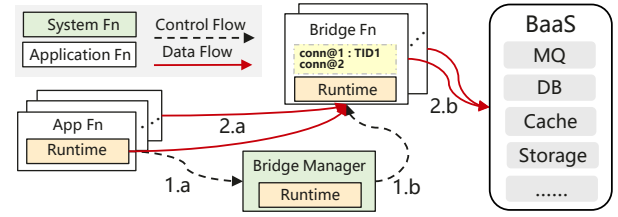
## 5 PORTABLE SERVICE BRIDGE

In this section, we describe a portable Service Bridge that provides a unified interface to support multi-cloud portability, and simplifies interactions among stateless and stateful.

### 5.1 Bridging Unified API & Diversified BaaS

Service Bridge serves as an extensible, portable framework implemented using YuanRong functions, as illustrated in Fig. 7, and inherently exhibits serverless characteristics such as rapid development, deployment, and automatic scaling. It allows for the installation of plug-in *bridge functions* by either application developers or cloud providers. Each bridge function incorporates a pool of BaaS connections, intercepts application requests and translates them into specific requests tailored to the required backends. As the volume of user requests increases, the number of bridge instances automatically scales up to handle the incoming requests. To optimize cost efficiency and track the lifecycle of all bridge instances, we developed a *bridge manager* that exists as soon as YuanRong is deployed and is automatically restored in the event of failure. The bridge functions are also automatically restored by the bridge manager following a failure. In such cases, the stateful connections handled by that bridge function are re-established automatically. Note that any operations that were in progress during the failure need to be re-executed by submitting a new application request.

Service Bridge provides a comprehensive set of unified interfaces by decoupling the implementation and interfaces of diversified BaaS, such as SQL databases. This approach eliminates the need for developers to integrate multiple SDKs with varying specifications when accessing BaaS across different cloud providers. Currently, we have focused on unifying the core data interface of BaaS, while leaving the management and configuration interfaces aside. Therefore, developers should continue to use the existing management interfaces to configure BaaS before running their application. For instance, the core interfaces of databases encompass data modification and transactions. As depicted in Table 1, we have abstracted the *Execute* interface for executing SQL operations and the *Commit* and *Roll-back* interfaces for transactions. Bridge API is also compatible with industry-standard interfaces, such as the JDBC interface. We encapsulate the Bridge API in an implementation of the JDBC interface, allowing developers to connect to the database via Service Bridge by simply adding the *CreateBridge* operation to the application code. By providing this unified backend interface abstraction, we enable seamless support for multi-cloud portability.



**Figure 7: Architecture of Service Bridge. All components are implemented by YuanRong functions.**

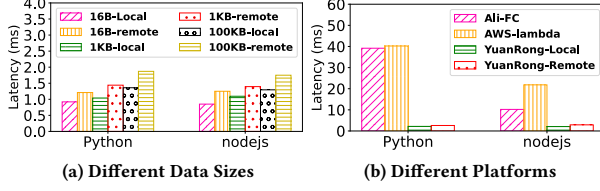
Service Bridge also offers a collection of pre-built bridge functions that act as proxies for requests originating from the unified interface. These bridge functions incorporate advanced capabilities that are not directly supported by backend services. For instance, the Cache’s *DELETE* operation supports the deletion of multiple keys, which Redis also supports but Memcached does not. To address this disparity, we implement a Memcached bridge function that leverages a loop to invoke the delete method, thereby abstracting away the differences between backend services. Meantime, application developers may require interfaces other than those provided by the unified interfaces. In such cases, developers can customize their interfaces by implementing lightweight bridge functions that cater to their unique requirements.

### 5.2 Bridging Stateless & Stateful

In our practical experience, the challenge arises when attempting to share connection states of BaaS among collaborating function instances, particularly when these instances operate concurrently. This predicament results in prolonged startup latency and inconsistent multi-function transactions.

To address this issue, we implemented a strategy that leverages a minimal number of bridge instances responsible for maintaining connections to BaaS. Developers can access a BaaS by invoking the *CreateBridge* interface (refer to Table 1) to request a bridge instance from bridge manager (Fig. 7 step 1.a) and the authentication configurations of the backend service is passed to the bridge manager. The bridge manager determines whether to scale or reuse an existing bridge instance based on references to currently available instances (Fig. 7 step 1.b). Bridge functions then open authenticated connections to BaaS (Fig. 7 step 2.b), converting the direct access to the BaaS into a function-to-function call (Fig. 7 step 2.a). By this way, connections to BaaS can be reused as well, resulting in following advantages.

**Enable Connection Reuse.** A practical concern arises when the number of requests to the BaaS gradually increases, potentially reaching the connection limit and rendering the service unavailable. Our bridge manager offers the capability to map requests from multiple user functions to the same bridge instance, facilitating the aggregation of a large number of requests to support high concurrency. Additionally, by integrating a connection pool into the bridge instances, we can circumvent the adverse effects of frequent establishment and termination of connections with the BaaS. This approach ensures that connections to the service with connection limitations remain within a specified threshold,



**Figure 8: Cross-function invocation latency of different platforms and data sizes with two programming languages.**

preventing the backend service from becoming unavailable. Alternatively, it also eliminates the need to re-establish connections to BaaS, consequently expediting the application startup process.

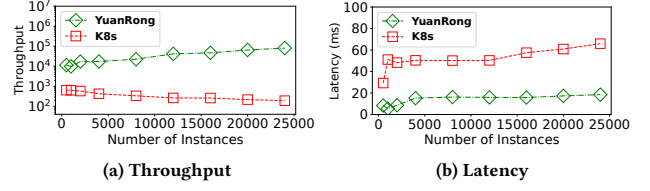
**Support Distributed Transactions.** It is common for our customers to perform concurrent operations on a backend service by launching a group of user functions. For example, when performing multiple operations to the same database, these operations need to be guaranteed to be transactionally consistent. Unfortunately, when these concurrent operations are performed by different user functions, this leads to distributed transactions and causes inconsistencies. To address this, we enable the functions to request a bridge instance from the bridge manager and subsequently call the bridge instance to initiate a transaction. The bridge function utilizes distinct backend database connections based on the same transaction ID to execute SQL and transactions. Once the transaction is complete, the user function performs a commit or rollback operation. As a result, Service Bridge ensures transaction consistency by converting distributed transactions into local transactions while still benefiting from the functionality of independent transactions. Due to page limitations, please find the details in § A.4.

## 6 EVALUATION

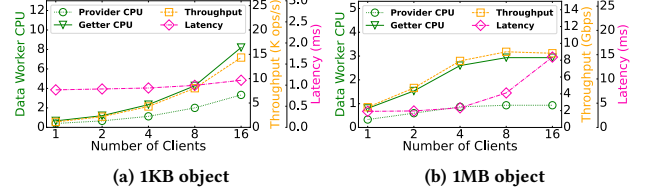
In this section, we begin by employing micro-benchmarks to verify the effectiveness of three key components of YuanRong. This validation process is crucial as the development journey from individual components to a fully integrated end-to-end system entails substantial effort and incurs trial-and-error costs. Subsequently, we present the experimental results obtained from large-scale runs of full-fledged applications within the production deployment.

### 6.1 Effectiveness of Function System

**Latency of Function Invocation.** The cluster we tested consists of 8 nodes, each with 16 CPUs and 32 GB of memory, and we evaluate the latency of invoking no-op functions without any payload. In our experiments, we report the average latency over 1000 runs, and all functions are warmed up to avoid cold starts on all platforms. Fig. 8a shows the invocation latency for different data sizes in YuanRong. We find that the latency for local function calls is nearly 1 ms, while the latency for cross-node function calls is within 2 ms for a data size of 100 KB. This demonstrates the superiority of function-to-function coordination through direct RPCs by minimizing components to avoid additional communication overheads as much as possible. Fig. 8b compares the invocation latency on three platforms with Python and nodejs languages. YuanRong performs significantly better than the others. For both languages, the latency on the Ali-FC and AWS-Lambda platforms is nearly 10–40× higher than YuanRong, regardless of whether the call is local or remote.



**Figure 9: The scheduling throughput and latency of YuanRong and K8s on a 3200-core CPU cluster.**



**Figure 10: Data worker CPU cost in object transfer.**

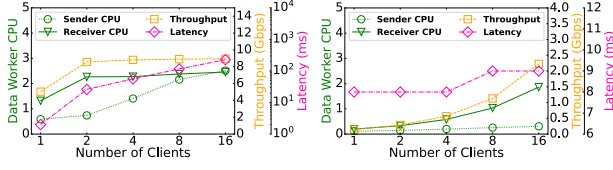
This is because both Ali-FC and AWS-Lambda rely on additional third-party services to achieve cross-function invocations, resulting in redundant paths and thus significant latency.

**Performance of Large-scale Scheduling.** We set up the cluster consisted of 200 nodes, each with 16 CPUs and 32 GB of memory. The number of scheduling groups is proportional to the number of scheduled function instances. Each group scheduler can handle about 4000 scheduling instances requests, we thus used a total of 7 scheduling groups. Figs. 9a and 9b show the scheduling throughput and latency as the number of instances increases, respectively. We observe that K8s's schedulers become the bottleneck under a high request rate and fail to quickly scale instances. While YuanRong's scheduling throughput increases linearly, and the scheduling latency is stable at 5–20 ms. This shows the effectiveness of YuanRong's distributed hierarchical scheduler.

### 6.2 Effectiveness of Data System

**Cost of Object Semantic.** We perform a cost analysis of object cache by measuring the cost of CPU and specifying the latency and throughput of data operations. We vary the number of clients from 1 to 16 and test different data sizes. Figs. 10a and 10b show the costs of the object provider and getter-worker CPU in obtaining 1KB and 1MB objects, respectively. The provider data worker requires less CPU cost than the object getter data worker (8 cores) because the getter data worker handles client-worker RPC requests and issues worker-worker RPCs. When the object size is 1 MB, the 10Gbps network bandwidth is saturated within 3 CPU cores. If the data size is 1 KB, the corresponding cost is about 8 cores due to RPC overhead. Nevertheless, we can optimize the transmission of small data by inlining values in function calls. Therefore, the cost is affordable for CPU. More detailed evaluation of object semantics about latency of object put/get, shared memory effect, scalability to number of nodes, and object spilling effect can be found in § A.5.1.

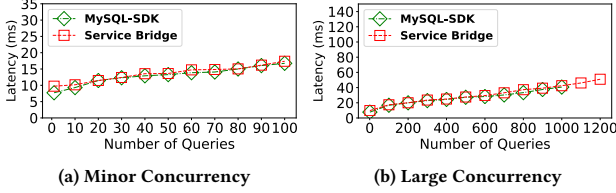
**Cost of Stream Semantic.** Figs. 11a and 11b show the cost of transferring stream elements with different input intervals. We vary the number of streams from 1 to 16 and each stream with a single producer and consumer. Input interval 0 means continuous sending without a pause, while interval 1 means a pause of 1 microsecond.



(a) 1KB element, 0us interval

(b) 1KB element, 1us interval

Figure 11: Data worker CPU cost in stream transfer.



(a) Minor Concurrency

(b) Large Concurrency

Figure 12: The average latency given different number of queries, comparing MySQL SDK to Service Bridge.

Results show that as the input data flow increases, the latency and throughput increase. When the workload is too large, the end-to-end element time is large due to the waiting time in the queue. We can achieve a latency of 2 milliseconds when the 10bps network bandwidth is almost saturated. The cost of CPU is also affordable and is less than 3 cores in all cases. We observe that the receiver side requires more CPU due to data movement and memory copies, while the data copies on the sender side are hidden via shared memory on the client side. More detailed evaluation about latency of stream data transferring can be found in § A.5.1.

### 6.3 Effectiveness of Service Bridge

We set up a cluster of 8 nodes and deploy MySQL 5.7.6 on another node under a different VPC, each with 16 CPUs and 32 GB memory. We create two user functions using MySQL SDK (setting the maximum number of database connections to 1000) and Service Bridge, respectively. We then run 1 to 1200 database queries simultaneously and record the average latency. Figs. 12a and 12b show that Service Bridge is comparable to MySQL SDK in terms of latency due to connection reuse, which eliminates the overhead of connection establishment, and sub-millisecond function invocation. As the number of queries increases and exceeds the maximum connection count of the MySQL server, the database is no longer accessible through the MySQL SDK, while our Service Bridge continues to work with little increase in latency.

We further adapted an application that uses database and cache services by replacing the corresponding SDK with Service Bridge. Experimental results show that Service Bridge reduced application startup latency from over 10 s to 24 ms, which is almost 400× faster. Meantime, memory usage was reduced from 26.4MB to 8MB, which is a significant reduction in resource consumption. This demonstrates the superiority of the unified BaaS abstraction and connection reuse capabilities provided by Service Bridge.

### 6.4 Effectiveness of Full-fledged Applications

**High-performance Computing.** Optical proximity correction (OPC) is one of the most computationally intensive workloads in semiconductor manufacturing. With the continuous shrinking of

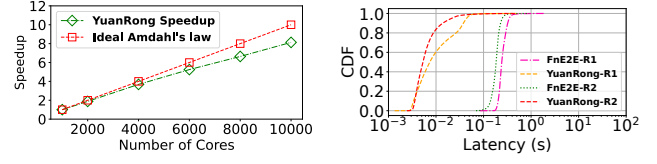


Figure 13: The scalability of YuanRong on OPC design under 10,000 cores.

Figure 14: The platform and function E2E latency under two regions.

semiconductor geometries, OPC entails executing billions of calculations across multiple mask layers to compensate for introduced distortions. Given the inherent complexity of this process, OPC necessitates the utilization of thousands of cores operating for extended periods to process a single semiconductor chip. The serverless paradigm, with its ability to scale compute resources infinitely, naturally meets the requirements of parallelizing this computation. To execute OPC, we employ a master-slave architecture in which a coordinator monitors and manages the workload and data. The coordinator dispatches individual compute jobs to workers, which receive data for specific portions of the mask, process the workload, and return the completed data to the coordinator. To evaluate the scalability, we start with 1,000 cores and gradually increased the number to 2,000, 4,000, 6,000, 8,000, and ultimately 10,000 cores for an M1 10mm × 6mm OPC design. Fig. 13 demonstrates that YuanRong successfully scales to the targeted 10,000 cores for a single design while maintaining nearly 80% scalability rate. This achievement can be attributed to YuanRong's hierarchical locality-aware scheduling mechanism, complemented by a built-in data cache that minimizes the overhead associated with frequent data movement.

**Enhanced FaaS Services.** FaaS applications typically involve the invocation of thousands of short-lived distributed functions to accomplish complex tasks that require burst parallelism. For example, one of our enterprise customers developed an attendance application that involves end users and IoT devices, necessitating real-time responses with a P99 end-to-end latency of less than 200ms. Figure 14 presents a comparison of platform and function end-to-end latency across two production regions. Region 1 (R1) and Region 2 (R2) comprise 900 and 10,000 CPU cores, respectively, and process 0.12 and 1.42 billion traffic per day. We observe that 50% of functions execute within 240ms, and nearly all functions complete within 2 seconds. Notably, the platform overhead remains below 35ms (P90 latency) in both regions, accounting for less than 15% of the lightweight function execution time, which is acceptable for most FaaS customers. We also observe that R1 exhibits longer average platform latency compared to R2, primarily due to the higher incidence of cold starts resulting from lower function invocation frequencies. The scalability of YuanRong, demonstrated by the comparable P99 platform latency at different cluster scales, can be attributed to the hierarchical scheduling mechanism. More detailed evaluation of micro applications such as streaming analytics and batch processing can be found in § A.5.2.

## 7 EXPERIENCES

In this section, we present some experiences and valuable lessons learned from the last 3 years during the rollout of YuanRong to our internal and external customers.

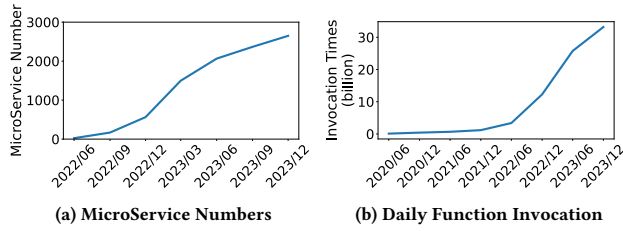


Figure 15: The evolution of YuanRong after deployment.

## 7.1 Deployment Platform Statistics

YuanRong has evolved through three stages, from stateless FaaS to native stateful support, and to the production general-purpose serverless platform. YuanRong has been widely deployed at Huawei in nearly 20 regions across China, Europe, Asia Pacific, etc., and provides internal cloud-native services and also an enhanced FaaS service for outside customers since the middle of 2020. YuanRong has been operational for roughly 1,400 days. Fig. 15a shows a nearly 110 times increase in the number of microservices migrated to YuanRong over the past 1.5 years. The substantial growth observed at the end of 2022 can be attributed to the successful pilot project involving the migration of business IT applications to YuanRong.

Fig. 15b shows the number of daily function invocations in YuanRong has increased 318 times over the past 3 years. This rapid growth can be attributed to multiple factors. On one hand, several high-volume internal microservices, such as the APIG microservice, with over 1 billion daily invocations, migrated to YuanRong. On the other hand, partnerships with external enterprises, including Toyota China, also contributed billions of daily invocations to our platform. By analyzing the statistics of function invocations, we observed significant variation across functions, spanning up to 9 orders of magnitude. Some functions experience over 1 billion invocations per day, while a mere 1.1% of functions account for 99.3% of all invocations, and these top-ranked functions exhibit strong periodicity. This insightful observation motivated us to explore the utilization of AI models to predict the top-ranked functions rather than all functions, enabling us to optimize instance allocation and improve platform cost-efficiency. Fig. 16 illustrates the CPU utilization and top-5 invocation times of functions within a China production region over a 24-hour period. We can see that after the deployment of YuanRong, the average used number of nodes is reduced 50% through rapid elasticity, compared to previous over-provisioning (i.e., reserving sufficient resources according to bursty in advance). This reduction in node usage translates to a 50% cost savings for customers. Meantime, the daily CPU resource utilization has increased 2.5 $\times$  than before by minimizing idle nodes, which fully demonstrates the superiority of the overall design of YuanRong. To date, YuanRong has served tens of thousands of enterprise customers and covered a large spectrum of workloads in our cloud, ranging from FaaS services, complex microservices, data analytics, to deep model training and serving, processing up to 30 billion requests per day on more than 100,000 CPU cores, achieving a daily average CPU usage of 53%.

## 7.2 Migrating Microservices to Serverless

In the summer of 2022, we started to help internal customers to change their microservice-based applications to serverless based on

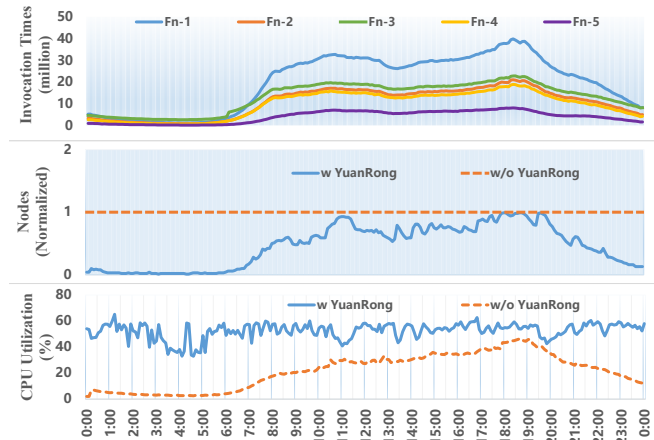
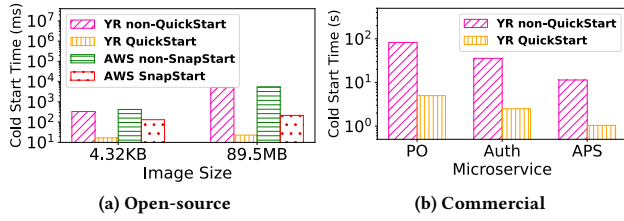


Figure 16: The top-5 invocation times of functions, number of used nodes, and CPU utilization within a China production region after YuanRong deployment over a 24-hour period.

YuanRong. The primary motivation behind migrating applications from microservice to serverless is to increase resource utilization, e.g., the CPU usage was around 5% in average, so to save cost for customers, and also to reduce the time-to-market (TTM) by removing tedious and error-prone work of managing computing infrastructure. However, we found there were several important challenges should be carefully addressed before this serverless migration could really happen.

The first challenge we encountered was many customers are hesitant to make significant changes to their code. This hesitancy was primarily driven by concerns about the need to re-debug well-tested code and the potential introduction of code compatibility issues, which could adversely impact the maintainability of their applications. Fortunately, we discovered that most of our customers relied on the Spring framework for constructing their microservice-based applications. To address their concerns, we build a custom Spring framework that incorporates two key changes. First, we devised an adapter that converts the incoming *HttpTriggerEvent* request into the corresponding *HttpServletRequest* based on parameters provided by the customers, and routes it to the appropriate *Servlet* for execution. The resulting *HttpServletResponse* from the *Servlet* is converted back into an *HttpTriggerEvent* and returned to the caller. Second, we replaced the standard *HttpClient* with our own custom *YuanrongClient* and wrapped the implementation of Spring's native key classes. These modifications enable our customers to continue utilizing existing Spring interfaces while intercepting requests originating from Spring's native *RestTemplate* or *Feign* interfaces and transforming the *Http* requests into function invocations within YuanRong. For other types of microservices, it is also possible to facilitate the serverless migration by substituting the components accountable for request handling and transmission.

After customers migrated their microservices-based applications to YuanRong as whole functions, they found limited improvement in resource utilization. This was primarily because the elasticity of the entire application, at its current granularity, still consumed the minimum reserved resources during periods of lower demand. Therefore, we encountered the second challenge that the microservice-based



**Figure 17: Cold-start time of YuanRong QuickStart and AWS SnapStart under open-source and commercial microservices.**

applications needed to be split into fine-grained functions to further increase resource utilization. Fortunately, we observed that our customers' microservice-based applications were typically composed of multiple loosely coupled, single-purpose jobs that were well-modularized. This offers an opportunity for us to analyze the call frequency of each job within the application. By identifying jobs with high-requested calls, we split them into independent functions, achieving fine-granularity elasticity and reducing resource waste. Concurrently, jobs with low-requested calls were packaged together into a single function, reducing the complexity and effort required for our customers to split their applications.

The third challenge we encountered when those fine-grained functions, which were split from the microservice-based applications, performed multiple operations on the same database. In this scenario, transactions inconsistencies could occur due to each function establishing a separate connection to the database for transaction operations. The potential for inconsistencies increased when one of the functions encountered a failure. Fortunately, our Service Bridge is well suited to solve this challenge by launching a bridge instance that proxy these database operations, converting distributed transactions into local transactions, and thus ensure transaction consistency by leveraging the functionality of independent transactions (see § 5.2 for details).

Besides, we encountered the fourth challenge related to long cold starts, which made on-demand function scaling impractical such as zero-scaling. Fortunately, we observed that high-requested functions exhibit strong daily periodicity, as they were user-facing and influenced by the human diurnal cycle. We thus leveraged AI models to predict the number of function invocations for the entire subsequent day [11, 30]. These predictions were then used to pre-emptively allocate an appropriate number of instances to serve a particular function, reducing the number and time of cold starts. For those low-requested functions which exhibit aperiodic patterns, we introduced an enhanced process-based snapshot (CRIU [22]) approach called QuickStart to reduce cold startup latency. As shown in Fig. 17a, YuanRong QuickStart achieves 6.2× and 8.8× speedups over AWS SnapStart w.r.t Java microservices with image sizes of 4.32KB and 89.5MB, respectively. This is because we use a process-level snapshot, which results in a smaller-sized snapshot file compared to the VM-level snapshot used by AWS. We also test three representative complex Java microservices in our business IT systems. Fig. 17b shows that QuickStart reduces application cold start latency by 10–16× compared to primary start without any optimization, benefiting from the skipping of time-consuming application initialization by restoring directly from the memory snapshot.

This serverless migration has helped our customer's service automatically scaled to a peak load of 50,000 transactions per second, with only a marginal increase of 0.1ms in average access latency compared to light load conditions. During a major promotional event on November 11, 2022, where daily requests surged to 2.7 billion, the resource costs were reduced by almost 90% compared to their previous approach of over-provisioning resources based on peak demand. Alternatively, by adopting a serverless model for our business IT applications, we have experienced significant simplification in development, such as eliminating resource preparation and operational overhead, reducing the time-to-market from 94 man-days to 30.5 man-days compared to traditional microservice architecture. Till now, close to 60% of microservices from tens of departments have seamlessly migrated to YuanRong, most of which can be easily done within 1 day. We plan to achieve 100% migration of microservices to YuanRong by the end of 2024.

## 8 RELATED WORK

Since the launch of AWS Lambda [35], interest in serverless has grown rapidly, and all major cloud providers now have similar FaaS offerings [18–20]. However, these offerings impose non-negligible restrictions such as maximum execution times, resource limits, etc., resulting in their predominant usage for simple event-driven applications (§ 1). In contrast, YuanRong liberates these restrictions by employing several technical innovations, such as function lifecycle management (§ A.2), for supporting general-purpose applications. Meanwhile, there are also a number of open-source serverless platforms [34, 39, 40]. Although some of the performance issues could be overcome with some custom optimizations, they have not yet proven themselves for large-scale production use and are thus not being adopted by cloud providers. Several industry platforms also offer serverless BaaS features and user experiences [6, 12, 13, 55]. However, these offerings are complementary to YuanRong as we provide a Service Bridge that can easily connect them to our platform, catering to general-purpose application requirements.

To extend the scope of serverless to more general workloads, there have been numerous efforts [1, 4, 5, 7, 9, 21, 25, 36, 37, 41, 44, 45, 48, 51–53, 57, 58, 61], but they still have some differences with the YuanRong design. First, our data system is different from existing cache and storage techniques. Pocket [33] adopts a disaggregated storage approach for serverless, where a separate pool stores key values or data objects, while we co-locate data with compute to achieve high performance under scalability consideration. Faasm [50] and Cloudburst [54] rely on a second-level cache to transfer data across nodes, while our data workers form a distributed memory pool, share and cache data, and synchronize metadata to provide lower latency and stronger consistency. Kafka [31] targets streaming storage via RPC-to-broker, while we support faster streaming processing via shared-memory-based data transfer with lower network overheads.

Second, we support low-latency, fine-grained function coordination through direct RPCs by minimizing components and simplifying cross-function invocation path, which is different from existing FaaS platforms [19, 20, 35] that require external services and thus introducing significant latency. While Nightcore [26] also makes most communication local for achieving low latency without going

through the API gateway, but it is designed for stateless mid-tier services and does not support stateful microservices due to lack of state management. Third, while Ray [38] and Pheromone [60] also adopt a two-tier scheduling architecture and consider data locality to improve performance. The major difference is that compared to their static sharding mechanism, our group scheduler forms a tree structure that is more flexible and dynamic to support the needs of large clusters while avoiding some cross-rack communications. Finally, our Service Bridge differs from Dapr [10], where sidecars are typically used to interact with BaaS, which has the disadvantage of fixed resource allocation. By making our Service Bridge serverless, we enjoy the rapid elasticity for high resource utilization and the flexibility to quickly develop new bridge functions.

Although these arts improve certain aspects of serverless, none of them has a holistic design for a general-purpose serverless platform. In theory, the above specialized systems [10, 50, 54, 60] could be combined to provide overall capabilities to support general-purpose applications, but in practice this results in not only low performance (e.g., massive data movement between systems), but also low development efficiency (e.g., multiple specific programming interfaces for developers). As a result, researchers and practitioners today typically build one-off serverless systems targeted at specific workloads [3, 8, 16, 17, 29, 42, 46, 49, 56]. In contrast, YuanRong provides a complete platform for general-purpose serverless by offering multi-semantic built-in data system for not only state management but also efficient data exchange, cooperated with a fast function system and an extensible portable Service Bridge framework.

Ray [38] has its own programming model and is more specific to ML applications, while YuanRong is a general-purpose serverless platform supporting microservices (§ 7.2) and various scenarios (§ 6.3). While distributed future semantic is a common technique [59], the biggest difference is that: (a) Ray places the object directory at runtime without considering the reliability of directory metadata, while YuanRong can synchronize the directory metadata and object values with reliable storage and recover the object values in case of failures; (b) Ray does not provide mutable object semantics or object consistency models; (c) Ray does not support multiple isolated sandbox technologies; (d) Ray lacks ability to access BaaS in a unified way, as provided by YuanRong Service Bridge.

## 9 DISCUSSION

**System Fault Tolerance.** We employ a lightweight technique for transparent recovery that relies on lineage reconstructions to prevent inconsistencies in failure scenarios. Initially, we record all the lineages of the function and asynchronously store them in our data system to optimize performance. The unsaved lineages are then transmitted to the callee node alongside the function call request, enabling the callee node to retrieve and recover them in the event of a failure by the caller. In cases where multiple function instances fail simultaneously, we also support parallel recovery by automatically resolving dependencies while replaying the lineage. When data object created by functions are crashed, the functions that get that object for usage will also crash. Our fault handling logic follows the lineage construction. To avoid lineage lost, we adopt write-through semantic of data objects to persist lineages in our data system. Note that in the event of an actor failure within

the system, we utilize the checkpoint/restore mechanism to ensure the state consistency of the restarted actors.

**Isolation and Security.** We incorporate an Identity and Access Management (IAM) module to authenticate requests and prevent unauthorized access between tenants, thereby ensuring secure logical isolation for multi-tenancy. When requests are transmitted within YuanRong, function instances query the IAM module to obtain an authentication token, which allows them to invoke other function instances or access data systems. To optimize subsequent requests from the same function instance, the token is stored in the cache of the local daemon process, thereby eliminating the need for additional authentication invocations and reducing latency. Furthermore, to enforce network isolation, each function instance initializes *iptables* to restrict network traffic, allowing tenant functions to exclusively access specific network segments. Lastly, YuanRong offers support for secure sandbox technologies, such as Kata Container and gVisor, which provide enhanced isolation guarantees.

**Heterogeneous Hardware Support.** We are enhancing YuanRong to efficiently manage heterogeneous hardware accelerators, supporting the Large Language Models (LLMs). First, we are extending our function system to support topology-aware scheduling and gang-scheduling of heterogeneous functions. Second, we are extending our multi-semantic data system by managing device high bandwidth memory (HBM) and leveraging the linked bus among these HBMs to achieve high-throughput data movements.

**Cloud Provider Cost.** Large-scale workloads in our cloud platform reveal that the resource utilization of serverless functions is much lower than the user requested resource limit. Based on this insight, we propose to leverage underutilized distributed memory to form a pool and deploy a *data worker* in each node to cache application intermediate data for better performance, achieving a customer-provider win-win deal. As for bridge functions in Service Bridge, customers are charged on demand while enjoying convenience, without adding cost to the provider. We left the exploration of a new serverless cost model as an interesting future work.

## 10 CONCLUSION

In this paper, we build the first production general-purpose serverless platform called YuanRong consisted of a general programming interface, multi-semantic data system, fast function system, and portable bridge system, to benefit various workloads in the cloud. YuanRong has been widely deployed at Huawei in nearly 20 regions, and served tens of thousands of enterprise customers, and covered a large spectrum of workloads, ranging from regular FaaS services, stateful microservices, data analytics, deep model training and serving, processing up to 30B requests per day on more than 100,000 CPU cores with 53% daily average CPU usage. While some design elements have been tailored to our particular needs, we believe many of them could also be inspiring for other serverless systems.

## ACKNOWLEDGMENTS

This paper presents several years of work by the YuanRong team at Huawei. We thank each team member for their contributions and all reviewers, especially our shepherd Guyue Liu, for their insightful feedback that has helped significantly improve the quality of paper.

## REFERENCES

- [1] Istemi Ekin Akkus, Ruichuan Chen, Ivica Rimac, Manuel Stein, Klaus Satzke, Andre Beck, Paaraja Aditya, and Volker Hilt. 2018. SAND: Towards High-Performance Serverless Computing. In *Proceedings of USENIX ATC*.
- [2] Lixiang Ao. 2022. *Framework and Platform Support for General Purpose Serverless Computing*. Ph.D. Dissertation. University of California, San Diego.
- [3] Lixiang Ao, Liz Izhikevich, Geoffrey M Voelker, and George Porter. 2018. Sprocket: A serverless video processing framework. In *Proceedings of the ACM Symposium on Cloud Computing*. 263–274.
- [4] Lixiang Ao, George Porter, and Geoffrey M Voelker. 2022. Faasnap: Faas made fast using snapshot-based vms. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 730–746.
- [5] Vivek M Bhasi, Jashwant Raj Gunasekaran, Prashanth Thinakaran, Cyan Subhra Mishra, Mahmut Taylan Kandemir, and Chita Das. 2021. Kraken: Adaptive container provisioning for deploying dynamic dags in serverless platforms. In *Proceedings of the ACM Symposium on Cloud Computing*. 153–167.
- [6] BigQuery. 2023. *Google Cloud-Products-Data Analytics*. Retrieved August 25, 2023 from <https://cloud.google.com/bigquery>
- [7] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–15.
- [8] Joao Carreira, Pedro Fonseca, Alexey Tumanov, Andrew Zhang, and Randy Katz. 2019. Cirrus: A serverless framework for end-to-end ml workflows. In *Proceedings of the ACM Symposium on Cloud Computing*. 13–24.
- [9] Benjamin Carver, Jingyuan Zhang, Ao Wang, Ali Anwar, Panruo Wu, and Yue Cheng. 2020. Wukong: A Scalable and Locality-Enhanced Framework for Serverless Parallel Computing. In *Proceedings of ACM SoCC*.
- [10] Dapr. 2023. *Distributed Application Runtime*. Retrieved August 17, 2023 from <https://dapr.io/>
- [11] Luke Nicholas Darlow, Artjom Joosen, Martin Asenov, Qiwen Deng, Jianfeng Wang, and Adam Barker. 2023. FoldFormer: sequence folding and seasonal attention for fine-grained long-term FaaS forecasting. In *Proceedings of the 3rd Workshop on Machine Learning and Systems*. 71–77.
- [12] Databricks. 2023. *Data Lakehouse Architecture and AI Company*. Retrieved August 25, 2023 from <https://www.databricks.com/>
- [13] Amazon DynamoDB. 2023. *AWS-Products-Database*. Retrieved August 15, 2023 from <https://aws.amazon.com/cn/dynamodb/>
- [14] Etc. 2023. *A distributed, reliable key-value store for the most critical data of a distributed system*. Retrieved August 15, 2023 from <https://etcd.io/>
- [15] Apache Flink. 2023. *Stateful Computations over Data Streams*. Retrieved August 15, 2023 from <https://flink.apache.org/>
- [16] Sadjad Fouladi, Francisco Romero, Dan Iter, Qian Li, Shuvo Chatterjee, Christos Kozyrakis, Matei Zaharia, and Keith Winstein. 2019. From Laptop to Lambda: Outsourcing Everyday Jobs to Thousands of Transient Functional Containers. In *Proceedings of USENIX ATC*.
- [17] Sadjad Fouladi, Riad S Wahby, Brennan Shacklett, Karthikeyan Vasuki Balasubramaniam, William Zeng, Rahul Bhalaria, Anirudh Sivaraman, George Porter, and Keith Winstein. 2017. Encoding, Fast and Slow: Low-Latency Video Processing Using Thousands of Tiny Threads. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 363–376.
- [18] Huawei FunctionGraph. 2023. *Huawei Cloud-Products-Compute*. Retrieved August 15, 2023 from <https://www.huaweicloud.com/intl/en-us/product/functiongraph.html>
- [19] Google Cloud Functions. 2023. *Cloud Functions-Serverless Computing-Products*. Retrieved August 15, 2023 from <https://cloud.google.com/functions>
- [20] Microsoft Azure Functions. 2023. *Azure Functions-Compute-Products*. Retrieved August 15, 2023 from <https://azure.microsoft.com/en-us/products/functions/>
- [21] Phani Kishore Gadepalli, Sean McBride, Gregor Peach, Ludmila Cherkasova, and Gabriel Parmer. 2020. Sledge: A Serverless-First, Light-Weight Wasm Runtime for the Edge. In *Proceedings of ACM Middleware*.
- [22] Paul H Hargrove and Jason C Duell. 2006. Berkeley lab checkpoint/restart (blcr) for linux clusters. In *Journal of Physics: Conference Series*, Vol. 46. IOP Publishing, 494.
- [23] Joseph M Hellerstein, Jose Faleiro, Joseph E Gonzalez, Johann Schleier-Smith, Vikram Sreekanti, Alexey Tumanov, and Chenggang Wu. 2018. Serverless computing: One step forward, two steps back. *arXiv preprint arXiv:1812.03651* (2018).
- [24] Benjamin Hindman. 2023. *libprocess, a concurrent and asynchronous programming library*. Retrieved August 15, 2023 from <https://github.com/3rdparty/libprocess>
- [25] Zhipeng Jia and Emmett Witchel. 2021. Boki: Stateful Serverless Computing with Shared Logs. In *Proceedings of the ACM SoSP*.
- [26] Zhipeng Jia and Emmett Witchel. 2021. Nightcore: Efficient and Scalable Serverless Computing for Latency-Sensitive, Interactive Microservices. In *Proceedings of ACM ASPLOS*.
- [27] Jiawei Jiang, Shaoduo Gan, Yue Liu, Fanlin Wang, Gustavo Alonso, Ana Klimovic, Ankit Singla, Wentao Wu, and Ce Zhang. 2021. Towards demystifying serverless machine learning training. In *Proceedings of the 2021 International Conference on Management of Data*. 857–871.
- [28] Zewen Jin, Yiming Zhu, Jiaan Zhu, Dongbo Yu, Cheng Li, Ruichuan Chen, Istemi Ekin Akkus, and Yulong Xu. 2021. Lessons learned from migrating complex stateful applications onto serverless platforms. In *Proceedings of the 12th ACM SIGOPS Asia-Pacific Workshop on Systems*. 89–96.
- [29] Eric Jonas, Qifan Pu, Shivaram Venkataraman, Ion Stoica, and Benjamin Recht. 2017. Occupy the cloud: Distributed computing for the 99%. In *Proceedings of the 2017 symposium on cloud computing*. 445–451.
- [30] Artjom Joosen, Ahmed Hassan, Martin Asenov, Rajkarn Singh, Luke Darlow, Jianfeng Wang, and Adam Barker. 2023. How does it function? Characterizing long-term trends in production serverless workloads. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*. 443–458.
- [31] Apache Kafka. 2023. *An Open-source Distributed Event Streaming Platform*. Retrieved August 15, 2023 from <https://kafka.apache.org/>
- [32] Ana Klimovic, Yawen Wang, Christos Kozyrakis, Patrick Stuedi, Jonas Pfefferle, and Animesh Trivedi. 2018. Understanding ephemeral storage for serverless analytics. In *2018 USENIX Annual Technical Conference (ATC)*. 789–794.
- [33] Ana Klimovic, Yawen Wang, Patrick Stuedi, Animesh Trivedi, Jonas Pfefferle, and Christos Kozyrakis. 2018. Pocket: Elastic Ephemeral Storage for Serverless Analytics.. In *Proceedings of USENIX OSDI*. 427–444.
- [34] Knative. 2023. *Serverless Containers in Kubernetes Environments*. Retrieved August 15, 2023 from <https://knative.dev/docs/>
- [35] AWS Lambda. 2023. *AWS Lambda-Compute-Products*. Retrieved August 15, 2023 from <https://aws.amazon.com/lambda/>
- [36] Zijun Li, Linsong Guo, Quan Chen, Jiagan Cheng, Chuhao Xu, Deze Zeng, Zhuo Song, Tao Ma, Yong Yang, Chao Li, et al. 2022. Help Rather Than Recycle: Alleviating Cold Startup in Serverless Computing Through {Inter-Function} Container Sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. 69–84.
- [37] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. 2022. FaaSFlow: Enable Efficient Workflow Execution for Function-as-a-Service. In *Proceedings of ACM ASPLOS*.
- [38] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elilib, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. 2018. Ray: A Distributed Framework for Emerging AI Applications. In *Proceedings of USENIX OSDI*.
- [39] OpenFaaS. 2023. *Serverless Functions, Made Simple*. Retrieved August 15, 2023 from <https://www.openfaas.com/>
- [40] Apache OpenWhisk. 2016. *Open Source Serverless Cloud Platform*. Retrieved August 15, 2023 from <https://openwhisk.apache.org/>
- [41] Li Pan, Lin Wang, Shutong Chen, and Fangming Liu. 2022. Retention-aware container caching for serverless edge computing. *Proc. of IEEE INFOCOM, IEEE* (2022).
- [42] Qifan Pu, Shivaram Venkataraman, and Ion Stoica. 2019. Shuffling, Fast and Slow: Scalable Analytics on Serverless Infrastructure. In *Proceedings of USENIX NSDI*.
- [43] Taavi Rehemiägi. 2018. *Companies using Serverless in Production*. Retrieved August 15, 2023 from <https://dashbird.io/blog/companies-using-serverless-in-production/>
- [44] Francisco Romero, Gohar Irfan Chaudhry, Íñigo Goiri, Pragna Gopa, Paul Batum, Neeraja J. Yadwadkar, Rodrigo Fonseca, Christos Kozyrakis, and Ricardo Bianchini. 2021. FaaS: A Transparent Auto-Scaling Cache for Serverless Applications. In *Proceedings of the ACM SoCC*.
- [45] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. 2022. Icebreaker: Warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM ASPLOS*. 753–767.
- [46] Josep Sampé, Gil Vernik, Marc Sánchez-Artigas, and Pedro García-López. 2018. Serverless Data Analytics in the IBM Cloud. In *Proceedings of ACM/IFIP Middleware*.
- [47] Johann Schleier-Smith, Vikram Sreekanti, Anurag Khandelwal, Joao Carreira, Neeraja J. Yadwadkar, Raluca Ada Popa, Joseph E. Gonzalez, Ion Stoica, and David A. Patterson. 2021. What Serverless Computing is and Should Become: The next Phase of Cloud Computing. *Commun. ACM* 64, 5 (apr 2021), 76–84. <https://doi.org/10.1145/3406011>
- [48] Mohammad Shahradd, Rodrigo Fonseca, Inigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. *arXiv preprint arXiv:2003.03423* (2020).
- [49] Vaishaal Shankar, Karl Krauth, Kailas Vodrahalli, Qifan Pu, Benjamin Recht, Ion Stoica, Jonathan Ragan-Kelley, Eric Jonas, and Shivaram Venkataraman. 2020. Serverless Linear Algebra. In *Proceedings of ACM SoCC*.
- [50] Simon Shillaker and Peter Pietzuch. 2020. Faasm: Lightweight Isolation for Efficient Stateful Serverless Computing. In *Proceedings of USENIX ATC*.
- [51] Wonseok Shin, Wook-Hee Kim, and Changwoo Min. 2022. Fireworks: A fast, efficient, and safe serverless framework using vm-level post-jit snapshot. In *Proceedings of the Seventeenth European Conference on Computer Systems*. 663–677.

- [52] Arjun Singhvi, Arjun Balasubramanian, Kevin Houck, Mohammed Danish Shaikh, Shivaram Venkataraman, and Aditya Akella. 2021. Atoll: A Scalable Low-Latency Serverless Platform. In *Proceedings of the ACM Symposium on Cloud Computing*.
- [53] AWS Lambda SnapStart. 2023. *Improving startup performance with Lambda SnapStart*. Retrieved August 15, 2023 from <https://docs.aws.amazon.com/lambda/latest/dg/snapstart.html>
- [54] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Joseph E. Gonzalez, Joseph M. Hellerstein, and Alexey Tumanov. 2020. Cloudburst: Stateful Functions-as-a-Service. *Proc. VLDB Endow.* 13, 12 (July 2020), 2438–2452.
- [55] Azure Blob Storage. 2023. *Azure-Products-Storage*. Retrieved August 15, 2023 from <https://azure.microsoft.com/en-us/products/storage/blobs>
- [56] John Thorpe, Yifan Qiao, Jonathan Eyolfson, Shen Teng, Guanzhou Hu, Zhihao Jia, Jinliang Wei, Keval Vora, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2021. Dorylus: Affordable, Scalable, and Accurate GNN Training with Distributed CPU Servers and Serverless Threads. In *Proceedings of USENIX OSDI*.
- [57] Dmitrii Ustiugov, Plamen Petrov, Marios Kogias, Edouard Bugnion, and Boris Grot. 2021. Benchmarking, analysis, and optimization of serverless function snapshots. In *Proceedings of the 26th ACM ASPLOS*. 559–572.
- [58] Kai-Ting Amy Wang, Rayson Ho, and Peng Wu. 2019. Replayable execution optimized for page sharing for a managed runtime environment. In *Proceedings of the Fourteenth EuroSys Conference 2019*. 1–16.
- [59] Stephanie Wang, Eric Liang, Edward Oakes, Ben Hindman, Frank Sifei Luan, Audrey Cheng, and Ion Stoica. 2021. Ownership: A Distributed Futures System for {Fine-Grained} Tasks. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. 671–686.
- [60] Minchen Yu, Tingjia Cao, Wei Wang, and Ruichuan Chen. 2023. Following the Data, Not the Function: Rethinking Function Orchestration in Serverless Computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 1489–1504. <https://www.usenix.org/conference/nsdi23/presentation/you>
- [61] Wen Zhang, Vivian Fang, Aurojit Panda, and Scott Shenker. 2020. Kappa: A Programming Framework for Serverless Computing. In *Proceedings of ACM SoCC*.

## A APPENDIX

Appendices are supporting material that has not been peer-reviewed.

### A.1 Multi-language Runtime

In the following, we present the implementation of the simple code snippets depicted in Figure 2 based on language-specific features, including Python, Java and C++.

**Python.** As an interpreted language, Python has a powerful serialization tool – *cloudpickle* – to package code fragments and dynamically launched them at a remote destination through interpreted execution. We leverage the built-in decoration syntax sugar of Python to mix definitions of remote and local functions in a single file. This greatly simplifies the developer’s efforts to manage functions. In the example in Figure 2a, a remote function, named *add\_one()*, is defined by simply adding a decoration of *yr.invoke*. And the local function can invoke *add\_one()* by calling its *invoke* method (Fig. 2a Line 5-6). The *invoke* method returns a *future* reference in *objref*, which can be *get* later in Line 7.

**Java.** JVM has a *ClassLoader* mechanism to load a class bytecode on-the-fly. Since Java is an object-oriented language, we need to package an entire class into a bytecode image that could be sent for remote execution. This is done by an auxiliary class, named *FuncHandler*, within the YuanRong Java Runtime. Similarly, the invocation to a remote function returns a reference to a future object as shown in Fig. 2b.

**C++.** It is actually not uncommon for our customers to use C/C++ to build high-performance distributed applications. Unfortunately, C++ has very limited support for reflections. Therefore, we leverage the macro pre-processing and meta-programming ability to enable remote functions. Figure 2c illustrates such an example. As shown

in Line 5, macro *YR\_INVOKE* instructs the compiler to generate a separate shared library for the specified class. We also need to maintain a function name-to-symbol name mapping in Runtime, and when the remote receives a call request, it can look up this mapping to find the proper symbol name and dynamically load the shared library. Invocation and future operations with C++ are just similar to that of Java in YuanRong Runtime.

## A.2 Fast Function System

### A.2.1 Function Metadata Management.

Due to the highly dynamic nature of serverless applications, it is crucial to track all function instances in a scalable way. In our earlier version, we adopt a centralized service to manage all function metadata, e.g., etcd [14]. However, a centralized metadata service incurs several issues: First, the capacity of the centralized metadata service needs manual provision and it is pretty hard to predict this right, especially with burst traffic in a large serverless cluster. Second, the centralized service may introduce single-point failure of the cluster. While we could introduce redundancy to make the centralized service robust, it will add much costs. Finally, function metadata has a high access frequency, and accessing a centralized service usually needs across-rack communication, which adds significant communication latency.

Therefore, in YuanRong, we shift to a complete distributed metadata directory. Each node has deployed a local *metadata directory*. When a function instance is scheduled (§ 3.2) to a node, the YuanRong kernel will simply store the function metadata in the local metadata directory (also called the *home directory* of the function). To facilitate the later location of the directory, the node ID of the home directory is encoded into the function ID, and when the function is referenced, the home directory can be easily decoded from the function ID. We choose the home directory by the node where the function scheduling request is issued, i.e., the caller function location. This way, if the callee function is scheduled on the same rack as the caller, we avoid across-rack communications in its home directory look-ups. The benefit motivates us to place the caller and callee functions as close as possible.

### A.2.2 Function Lifecycle Management.

Once a function instance is created and running, it will process the requests from its request queue. But when the queue is empty, we mark the function as *idle*. We stop accounting function, but do not immediately reclaim the resource as long as there are enough spare resources in the node. This *lazy resource reclaim* helps greatly to reduce the number of cold-start as an idle function may soon become *active* when a new request comes in. When the allocated resource hits a threshold defined by the percentage of current node’s total resources (e.g., 80% in our system), a reclaim process is executed that changes *idle* function into *suspended*. A suspended function will lose its computing resource, but may still keep a copy of its internal state in the data system, which can be later used to resume the function. To save the function state, the *SaveState* method will be called. The application function should implement this method and store the proper internal state. When a function is resumed, a new instance is created and its *LoadState* method is called, where the new instance can recover the internal

state from the stored data. A *suspended* function will eventually become *terminated* due to 1) explicitly being terminated or 2) its caller function is already terminated.

### A.3 Multi-Semantic Data System

#### A.3.1 Object Semantic Data Cache.

**Shared Object Lifecycle.** To manage the lifecycle of shared objects, we provides interfaces to increase and decrease the reference count, and objects are automatically deleted when the reference count reaches zero. To minimize IPC overhead, we establish a hierarchy of reference counting between functions, data workers, and the object directory. Data worker and object directory maintain the number of functions and data workers referencing the object, respectively. IPC occurs only when the number of references changes from 0 to 1 or from 1 to 0. Moreover, an object may have dependencies on other references. Thus, we also store the reference dependencies on the object directory.

**Private State Lifecycle.** To manage the lifecycle of function private states, we employ an explicit deletion mechanism, as the state has the same lifecycle as the function itself. The state is cached in the data system and persisted to reliable storage until the function terminates. To maintain object reliability over an extended period, we support writing to a reliable storage such as OBS. Objects and direcotry metadata stored in reliable storage are deleted when they are no longer needed. In the event of a data worker crash, data can be retrieved from reliable storage.

**Object Reliability.** The local SSD is used as an extension of local memory for caching. To preserve object reliability and maintain it over a long period of time, we support writing to a reliable storage service, e.g., OBS, and the object in reliable storage is deleted when it is no longer needed. If a data worker crashes, we can retrieve the data from reliable storage. Directory metadata is also kept in reliable storage to ensure successful recovery. Our semantics of the function state is based on the reliability property. Programmers specify a write-through option in object putting and the reliability is ensured when the put operation returns.

#### A.3.2 Stream Semantic Data Cache.

**Backpressure Mechanism.** Due to variations in processing speed between producers and consumers, we propose a backpressure mechanism to avoid waste of busy-waiting and unnecessary data pushing. In our design, each stream on a node is associated with a shared memory capacity. When pages are fully utilized and have not yet been recycled, backpressure actions are triggered. For data transfer across nodes, the receiving node notifies the sending node to pause transferring, thereby optimizing the process. In the case of data transfer within a single node, the producer function can choose to block until a new page becomes available.

### A.4 Portable Service Bridge

#### A.4.1 Support Distributed Transaction.

When our customers perform concurrent operations on a backend service by launching a group of user functions (referred to as  $f_1$  and  $f_2$ ). To address inconsistencies in distributed transactions, we enable  $f_1$  to request a bridge instance from the bridge manager and subsequently call the bridge instance to initiate a transaction.

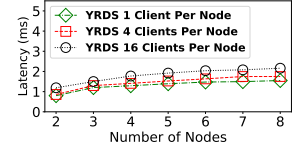
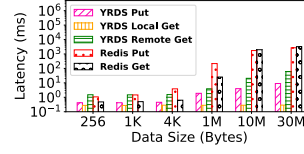


Figure 18: Average latency of object put and get.

Figure 19: Average latency of across-node object get.

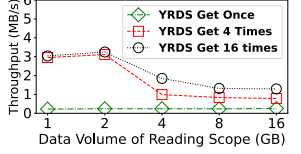
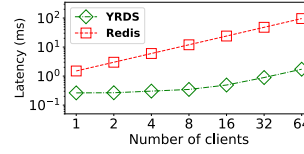


Figure 20: Average latency of single hot object get.

Figure 21: Throughput of hotness object get.

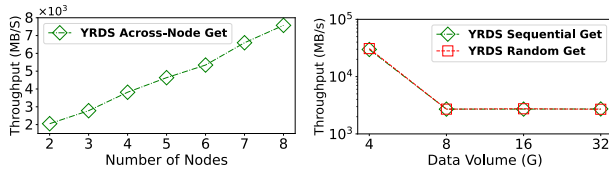
The bridge instance generates a unique identifier (ID) for the transaction and returns it to  $f_1$ . Subsequently,  $f_1$  utilizes this transaction ID when invoking the bridge function until the completion of the transaction. The bridge instance maintains a mapping between the transaction ID and a specific backend database connection that can only be used by requests associated with that transaction ID.  $f_1$  transfers the transaction ID and bridge ID to  $f_2$ , allowing  $f_2$  to send requests to the same bridge instance that contains the relevant transaction ID. The bridge function utilizes distinct backend database connections based on the transaction ID to execute SQL and transactions. Once the transaction is complete, the user function performs a commit or rollback operation. The bridge function then removes the mapping relationship between the transaction ID and the connection and returns the connection to the connection pool. As a result, Service Bridge ensures transaction consistency by converting distributed transactions into local transactions while still benefiting from the functionality of independent transactions.

Compared to establishing a direct connection with BaaS, our approach does not compromise the reliability of transactions. First, since transactions are serialized, the upstream function detects errors through inter-function calls when an exception occurs during a transaction. In such cases, the upstream function can initiate a rollback of the transaction by invoking the rollback interface to prevent the presence of inconsistent or erroneous data. Second, if an exception occurs across all user functions, such as reaching a timeout (which is user-configurable, with a default value of 5 minutes), the bridge function recycles the connection and automatically rolls back the transaction. Finally, if an exception arises within the bridge function itself, the connection to the BaaS is automatically terminated, and the transaction is also automatically rolled back. User functions can handle these exceptions similarly to when using a direct connection to BaaS, allowing for appropriate error handling and retries.

### A.5 Evaluation

#### A.5.1 Evaluating Data System.

**Latency of Object Put/Get.** We put and get objects on a single client, measure the average latency, compare YuanRong data system (YRDS) to a cloud deployed single-instance Redis, and show results



**Figure 22: Throughput of across-node object get.**

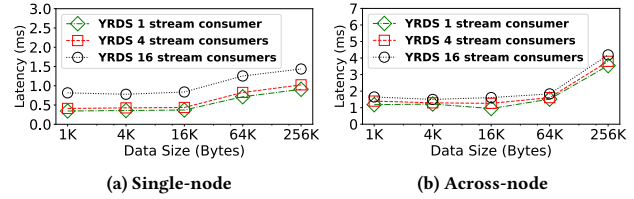
in Fig. 18. YRDS object putting is 2.5–299 $\times$  faster than that of Redis, because we always put objects locally, whereas Redis incurs network traffic overheads. YRDS local getting is 1.7–11076 $\times$  faster than that of Redis due to affinity-aware scheduling and shared memory zero-copy getting. For remote objects getting, the performance of YRDS under KB-scale data is slower due to the object directory overheads. Nevertheless, we have optimized by inlining small function argument in the function call and pass-by-value. For MB-scale data, YRDS is 6.5–53.7 $\times$  faster than Redis due to better throughput of the built-in cache.

**Effect of Local Cache.** We evaluate cross-node object getting latency in hot-key scenarios. We perform put operations for a set of keys and then get operations for the entire set on all nodes. We vary the number of nodes and change the workloads by varying the number of YRDS clients on each node. In Fig. 19, the latency does not increase linearly even though individual hot keys are accessed simultaneously. This is because we have a local cache on different data workers, and data workers can work with the cache as a data provider to balance the data access to a single key. When the workload increases from 1 to 16 YRDS clients, the increase in latency is within 40%, which shows that YRDS handles object access well with high concurrency.

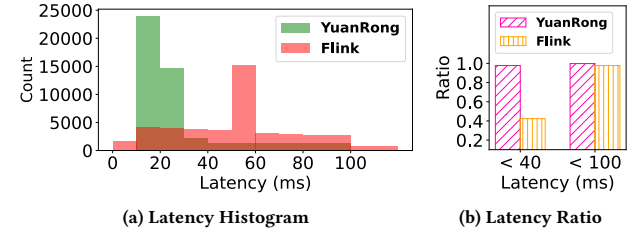
**Effect of Shared Memory.** We measure concurrent getting latency of a single-key, vary the number of clients on the same node and perform local-gets, and show the results in Fig. 20. As the number of clients increases, the latency of YRDS increases slightly due to inter-process metadata communication overheads, whereas the latency of Redis increases linearly due to sequential queue execution. This shows that Redis has a single-point bottleneck for hot-key object getting, while YRDS employs distributed memory sharing with multi-threading serving, and thus can handle hot-key objects well. The latency in Fig. 20 is much lower than in Fig. 19, because we use shared memory for get operations on the same node, while in Fig. 19 the data is transferred between nodes.

**Scalability to Number of Nodes.** We deploy data workers on 8 nodes. To saturate a data worker, we have 48 clients on each node. We perform put operations for all the data first and then get all the remote data from across-node workers. We vary the number of actively data transferring nodes and show the throughput in Fig. 22. The network bandwidth is 10 Gbps, and the results show that we can saturate the bandwidth and scale linearly to the number of nodes. This is because we have distributed object directory management and load balance techniques. Thus, both metadata and data serving does not become a bottleneck.

**Effect of Object Spilling.** We deploy a data worker on a single node, and set the memory capacity 4GB. We perform put operations of multiple keys and then get these keys from 16 clients. We vary the total amount of data volume from 2GB to 32GB, measure two



**Figure 23: Throughput of large volume object get.**



**Figure 24: Average latency of stream data transfer.**

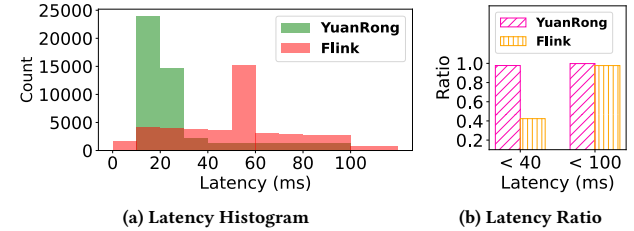
workload cases, and report the throughput in object getting. In Fig. 23, both curves have a decreasing slope and reaches close to I/O throughput. The result shows we can efficiently saturate the SSD bandwidth to 2600MB/s. This is because we have asynchronous flushing to SSDs and the bottleneck is I/O throughput.

**Effect of Caching for Hot Objects.** We keep the data worker and memory capacity settings and fix data volume of data at 32GB. We vary the hotness of data by changing the scope of random read elements, from 1GB to 32GB, and we measure the throughput for cases of same key reading in different times 1 $\times$ , 4 $\times$ , and 16 $\times$ . In Fig. 21, our throughput is up to 30.8GB/s when data is accessed frequently in a small scope. This implies when there are hot objects in a small range, we can also keep moderate performance because of the shared memory accessing.

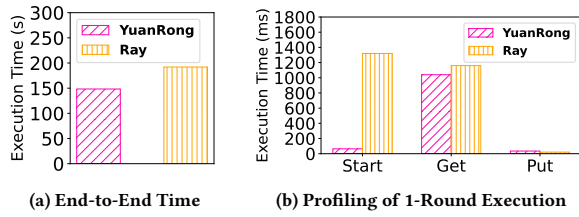
**Latency of Stream Data Transferring.** We conduct experiments with one and two nodes, respectively, varying the data size and the number of consumers for a stream and evaluating the latency in data transfer. Fig. 24 shows that increasing the number of consumers from 1 to 16 increases the latency within 50%. This is because we can reuse shared memory caching to reduce data movement and memory copying. As the data size increases, the latency increases especially for 64 and 256 KB data. This is due to memory copy and network overheads for single-node and across-node transferring, respectively. For small data (1KB, 4KB, and 16KB), the overheads mainly lies in the IPC and network traffic latency.

#### A.5.2 Evaluating Micro Applications.

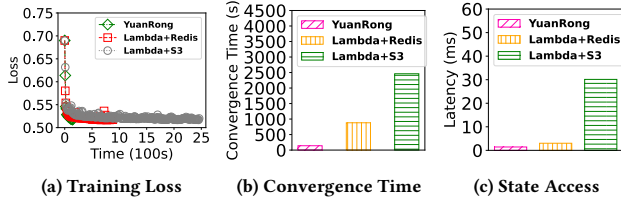
**Streaming Analytics.** We build an online financial analysis pipeline on YuanRong and compare its processing latency with Flink [15]. The pipeline consists of source and sink operators connecting Kafka and intermediate operators such as *map*, *keyby*, and *flatMap* to perform financial quantification. We launch a four-node cluster using YuanRong, Flink, and Kafka, and produce stock data per millisecond in Kafka. Fig. 25a shows that we have lower average latency than Flink (22.39 versus 49.23 ms) and the ratio of latency is better in small latency ranges, e.g., below 40 ms (see Fig. 25b). The improvements are due to a couple of reasons. First, shared memory



**Figure 25: Online financial analysis latency of YuanRong and Flink over 42,250 streaming stocks data.**



**Figure 26: Offline financial quantitative analysis execution time of YuanRong and Ray over 4,000 stocks.**



**Figure 27: The distributed training convergence time and its profiling. We use a SGD sparse logistic regression algorithm and set the maximum number of iterations to 30,000.**

page buffers can be directly used for serialization and shared by operators, whereas in Flink temporary byte buffers are generated during serialization and incur memory copy overheads in buffer transferring. Second, we schedule correlated functions on the same node, which further reduces the network traffic overheads and intermediate data transfer across nodes.

**Batch Processing.** We build an offline financial quantification business which involves batch processing. Ray is chosen as a comparison because it is widely used in financial companies such as Ant Financial [38] for similar purposes. The input is transaction data of 4,000 stocks in a securities firm for one year. We parallelize by invoking multiple YuanRong functions and coordinating them via the object reference semantics. Fig. 26a shows that end-to-end execution time of YuanRong is 77% of Ray. We further profiled the execution time of one round. As can be seen in Fig. 26b, YuanRong is 20× faster than Ray in terms of instance startup, because YuanRong’s QuickStart can significantly reduce the cold start time. The object *get* performance of YuanRong is better, while the *put* performance of Ray is slightly better.

**Distributed Training.** Machine learning is a typical workload for general-purpose serverless. To this end, we implement distributed training based on the parameter server architecture, using distributed SGD sparse logistic regression algorithm over Criteo dataset (i.e., selects 1 GB dataset of display ads) and the selected comparison baselines are Lambda+S3 and Lambda+Redis from PyWren [29]. In each iteration, model replica actors compute gradients in parallel, send the gradients to a shared parameter server, then read the summed gradients from the server for the next iteration. As shown in Fig. 27a, the model convergence rate on YuanRong is much faster than Lambda+Redis and Lambda+S3. Fig. 27b shows the convergence time of distributed training. YuanRong is 17.57× faster than Lambda+S3 and 6.28× faster than Lambda+Redis. This is because YuanRong provides a built-in data system, which greatly reduces state read and write latency compared with external state access. Fig. 27c further proves the above conclusion.